Two-page sample of my paper. This is chosen from “Service Package Recommendation for Mashup Creation via Mashup Textual Description Mining,” a paper accepted by The International Conference on Web Services (ICWS) 2016. I independently designed and implemented a service recommendation engine with a three-layer architecture, which had provided rich and stable supportive data for our theoretical part of study.

Figure 1 shows a service recommendation use case for mashup creation. When a user is going to create a new mashup application that can produce a start page with favorite contents, the user first describes the requirements as "A mashup developed to create a start page with your favorite contents. Import your photos, videos, music, bookmarks. View your favorite websites in one place. Discover similar contents. Collect, save and share your contents."

The recommendation engine firstly decomposes the mashup description into several sentence blocks. Using *the discourse parser tool*, the engine will also identify relationships between these sentence blocks. Then each sentence block will be matched with several services from the repository. Since semantic relationships among services in the repository have been learned from historical mashups and their descriptions, the recommendation engine will try to recommend a set of services whose relationships are consistent with the relationships among sentence blocks.

In this scenario, the recommendation engine will recommend *Flickr, del.icio.us, YouTube, Last.fm, Facebook, Twitter, LinkedIn, Thumbalizr* and *Hyves* to build the mashup application which presents the start page with favorite websites.

B. Problem Definition

Given the textual description D_m of a mashup m to be created, we parse the textual description D_m into statement blocks $S^{(m)} = \{s_1^{(m)}, s_2^{(m)}, \dots, s_{|S^{(m)}|}^{(m)}\}$, where $|S^{(m)}|$ is the number of statement blocks that the description decomposed into. Each statement block $s_i^{(m)}$ is related to a set of services $WS(s_i^{(m)}) = \{ws_i^1, ws_i^2, \dots, ws_i^{|WS(s_i^{(m)})|}\}$, where $|WS(s_i^{(m)})|$ is the number of services recommended to statement block $s_i^{(m)}$.

With the relationships exploited from the historical data, the service package $SP = \{ws_1, ws_2, \dots, ws_{|SP|}\}$ is recommended which can fulfil the user's need by maximizing some utility function, where $|SP|$ is the number of services in the service package.

IV. Framework and Algorithms

In this section, first, we present the framework of our proposed service package recommendation approach. Then we introduce discourse parser, which is a natural language tool to analyze the textual description of a given mashup. Finally, we describe the algorithms for service recommendation and service package recommendation for mashup development in detail.

A. Service Package Recommendation Framework

Figure 2 shows the framework for service package recommendation. It mainly consists of two parts:

1) Service Recommendation Engine

It receives the textual description of a mashup, which depicts the functional specification of the desired application. This specification is often written in natural language. A segmenter breaks the description into EDUs, which are the ***E***lementary ***D***iscourse ***U***nits in discourse analysis. Based on the EDUs, discourse parser links the EDUs into a labeled tree (Discourse Tree). Also, we use WordNet [17] to match EDUs with services' descriptions, which forms a large lexical database of English. Thus, we can recommend a list of services to each EDUs.

2) Service Repository

A service repository with enough historical mashup records is the knowledge base of our approach. We segment the description of each mashup into EDUs and construct a corresponding discourse tree. Based on the discourse analysis and LSI (Latent Semantic Indexing) model, we can generate the relation matrix of services.

B. Discourse Analysis

The functional description of a mashup is not merely a sequence of independent and isolated sentences, in contrast, the meaning of a sentence also relates to the adjacent sentences. The description exhibits a finer-level structure

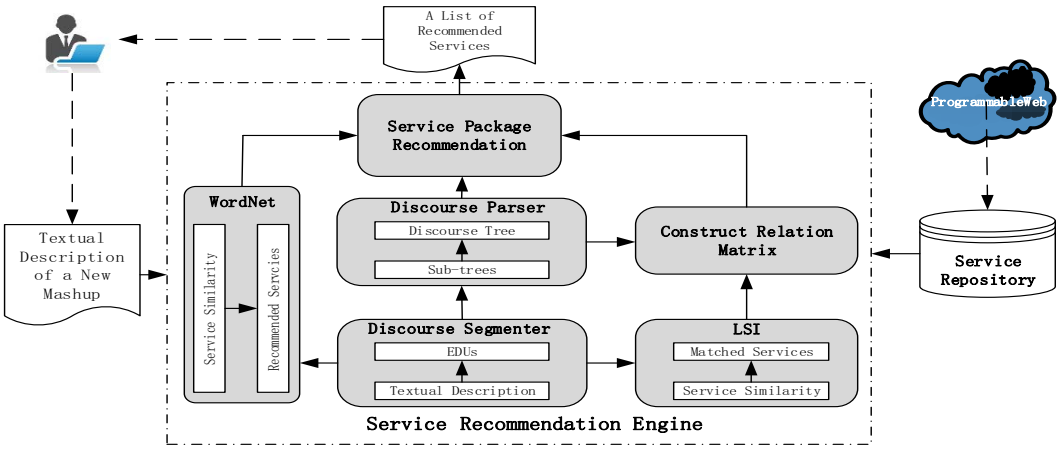

Figure 2. The Overall Structure of Our Framework

called coherence structure in which the clauses and sentences are logically bund together to express the meaning as a whole. Discourse analysis is well-studied in many natural language processing applications, which seeks to uncover the coherence structure between the related sentences.

In order to explore the coherence structure of the functional descriptions of a mashup so as to uncover the relationships between services, we utilize CODRA [18], which is a complete probabilistic discriminative framework for discourse analysis. CODRA uncovers the coherence structure under the text and posits a tree representation of a discourse. Thus, it represents the description by labeled hierarchical structures called Discourse Trees (DTs). Figure 3 shows the Discourse Tree for the use case mentioned in Section III.

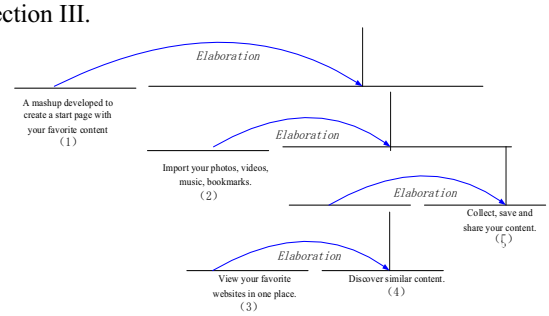

Figure 3. The Discourse Tree for the Use Case

The leaves of the DT correspond to the EDUs in the description. Horizontal lines indicate statement blocks. Adjacent EDUs are identified with coherence relations (e.g. Elaboration), they are connected by curved arrows with the identified relationships. CODRA comprises two parts: discourse segmenter and discourse parser.

1) Discourse Segmenter

Discourse segmenter breaks the functional description of a mashup into a sequence of EDUs. Actually, the task of discourse segmenter is to find EDU boundaries inside sentences. CODRA utilizes a maximum entropy model for binary classification. Currently, automatic discourse segmentation has attained accuracies close to human performances [19].

2) Discourse Parser

Discourse parser links the EDUs into DT. In the process of building the DT, discourse parser determines which discourse units to relate, and what relations to use when the EDUs are identified. Discourse parser comprises two probabilistic discriminative parsing models: intra-sentential parser and multi-sentential parser. Intra-sentential parser produces one or more discourse sub-trees for each sentence while multi-sentential parser generates a full discourse tree for the document from these sub-trees.

C. Construct Relation Matrix

Discourse analysis breaks the textual description into EDUs and constructs a discourse tree. In order to match each EDU to a service and learn the relations between services, we use LSI method, which tries to overcome the problem of lexical matching by using statistically derived conceptual indices instead of individual words for retrieval. In order to depict the correlations between keywords, we utilize the descriptions of all the mashups and services in our real-world dataset to construct the service repository. Then, we clear the data through word segmentation, removing the stop words, and stemming.

By utilizing the LSI model, we compute the similarity between EDUs and services. The service with the highest similarity value is the matched one. In order to simplify the mapping process, if the service name appears in the EDU we regard it as the matched service directly.

Based on the constructed discourse tree, we can then obtain semantic relations between services by inheriting the relationships between corresponding EDUs. At the same time, the co-occurrence relationships can be easily obtained from historical mashups:

a) Co-occurrence Relation

If two services are used by the same mashup, we say they have co-occurrence relationship. For example, mashup m is composed of service A, service B, service C, we say service A, B and C have a co-occurrence relationship. Co-occurrence is the elementary relation between services.

b) Semantic Relation

Based on the elementary co-occurrence relation between services, we discover several different semantic relations between services using the tool CODRA. Table I shows different relations discovered in our real-world dataset in detail. E.g. “sameblock” means two services belong to the same block, “count=1000” means the number of different service pairs have this semantic relation comes to 1000, whereas “sum=4330” means the total number of this semantic relation comes to 4330.

TABLE I. STATISTICS OF RELATIONS BETWEEN SERVICES IN OUR DATA SET

Name	Count	Sum
sameblock	1000	4330
comparison-comparison	4	4
topic-comment-topic-comment	2	2
temporal-temporal	46	46
span-condition	18	18
condition-span	18	18
attribution-span	142	154
span-attribution	142	154
enablement-span	254	267
span-enablement	254	267
manner-means-span	54	54
span-manner-means	54	54
span-contrast	13	13
contrast-span	13	13
joint-joint	1000	1274
background-span	107	109
span-background	107	109
span-cause	45	45
cause-span	45	45
span-elaboration	1000	1566
elaboration-span	1000	1566
same-unit-same-unit	690	752
span-temporal	16	16
temporal-span	16	16
explanation-span	17	17